

Technical Documentation: ANE2 Remote RF Sensor System

Engineering Team

February 10, 2026

Contents

1	System Overview	2
2	Software Architecture	2
2.1	System Daemons	2
3	The Orchestrator & System Logic	2
3.1	Global State Machine	2
3.2	Real-time Operational Loop	2
3.3	Campaign Management and Crontab Synchronization	3
3.4	Advanced Acquisition Strategies (AcquireDual)	3
3.4.1	DC Spike Removal	3
3.4.2	Spectral Stitching (Cosido)	3
4	System Monitoring and Status	4
5	System Initialization	4

1 System Overview

The ANE2 system is a remote RF sensor platform designed for high-performance data acquisition, signal processing, and scheduled measurement campaigns. It utilizes a Raspberry Pi as the core compute node to manage specialized hardware like the HackRF.

2 Software Architecture

The software is organized into a modular daemon-based architecture to ensure reliability.

2.1 System Daemons

The system is managed by several `systemd` units generated during the provisioning phase:

- **rf-ane2.service**: Core RF application for data acquisition.
- **orchestrator-ane2.service**: Central engine for state management.
- **status-ane2.service/timer**: Periodic health monitoring.
- **ltegps-ane2.service**: Telemetry and location data.

3 The Orchestrator & System Logic

The `orchestrator.py` module serves as the central cognitive unit of the ANE2 system. It is responsible for high-level decision-making, state transition management, and ensuring that hardware resources are utilized efficiently without task overlap.

3.1 Global State Machine

The system's behavior is governed by a formal state machine defined in the `SysState` enumeration. Transitions are managed by the `GlobalSys` controller, which enforces mutual exclusivity between tasks.

- **IDLE**: The default resting state. The system polls the API for new real-time instructions or scheduled measurement windows.
- **REALTIME**: Triggered by user demand via the API. In this state, the system provides a continuous stream of spectral data and optional demodulated audio.
- **CAMPAIGN**: Activated when a scheduled measurement window opens. The system executes high-priority tasks defined by the central server.
- **KALIBRATING**: A transient state where the system runs `kal_sync.py` to correct clock drift and frequency offsets before starting a campaign.
- **ERROR**: A safety state entered upon critical hardware or network failure.

3.2 Real-time Operational Loop

When in `REALTIME` mode, the orchestrator enters a "Sticky" execution loop. It performs a continuous heartbeat with the API to fetch `ServerRealtimeConfig` objects.

- **WebRTC Integration**: If demodulation (AM/FM) is requested, the orchestrator spawns a `server_webrtc.py` subprocess to handle low-latency audio distribution.
- **Dynamic Configuration**: The loop monitors changes in center frequency, sample rate, and gains. If a demodulation change is detected, it sends a synchronization command to the RF Engine via ZMQ to reset the DSP pipeline.

3.3 Campaign Management and Crontab Synchronization

The `CronSchedulerCampaign` class manages the persistence of measurement tasks. Unlike real-time tasks, campaigns are scheduled at the OS level to ensure reliability.

- **Priority Logic:** If multiple campaigns overlap in the same time window, the scheduler applies a "Winner-Takes-All" strategy, selecting the campaign with the highest `campaign_id`.
- **System Integration:** Valid campaigns are converted into standard Linux `cron` entries with the comment prefix `CAMPAIGN_[ID]`.
- **State Persistence:** Campaign parameters are stored in the `ShmStore` (Shared Memory), allowing the `campaign_runner.py` to access hardware settings even after a system reboot.

3.4 Advanced Acquisition Strategies (AcquireDual)

To mitigate artifacts inherent in low-cost SDR hardware, the orchestrator utilizes the `AcquireDual` engine. This component implements two primary correction strategies:

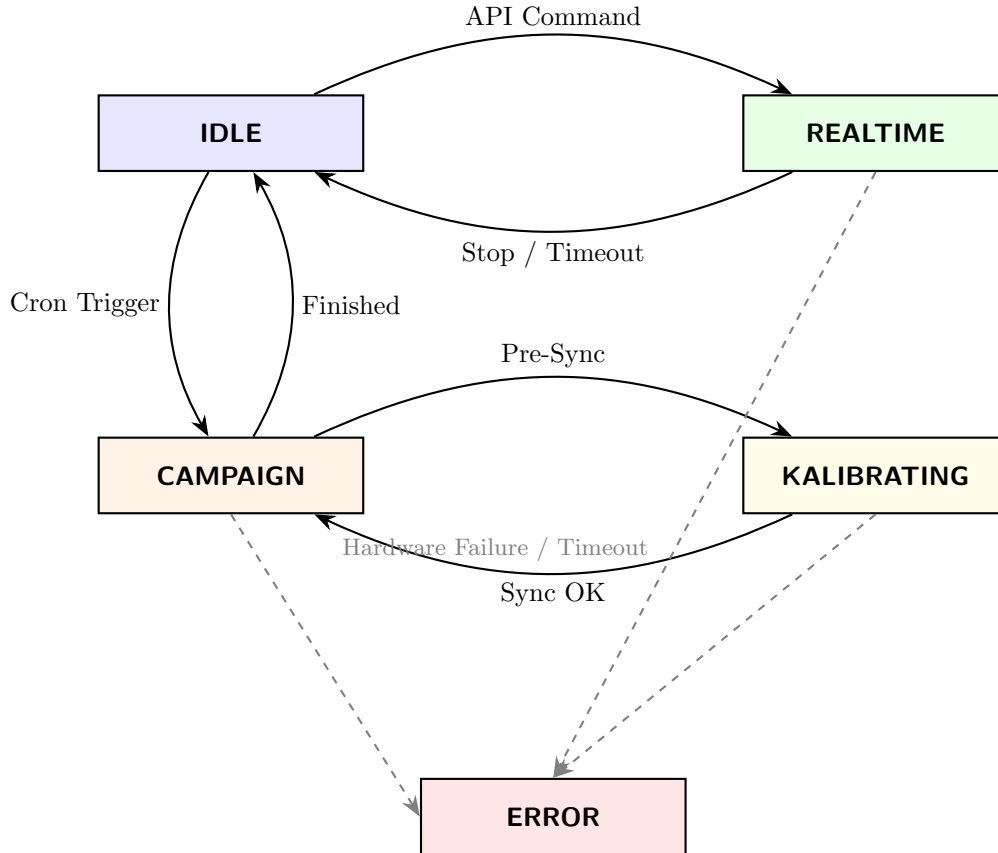
3.4.1 DC Spike Removal

The "Center Spike" artifact is removed by sampling the noise floor in the immediate vicinity of the center frequency. The engine then injects synthetic Gaussian noise into the central 0.2% of the bins, matching the local mean and standard deviation (μ, σ) to maintain spectral integrity.

3.4.2 Spectral Stitching (Cosido)

For high-precision captures, the system performs a dual-acquisition "Stitching" technique:

1. Capture A is taken at the desired center frequency (f_c).
2. Capture B is taken with a frequency offset (e.g., +2 MHz).
3. A linear correction ramp is calculated to align the amplitudes of both captures.
4. A cosine-faded blending mask is applied to merge the clean data from Capture B into the distorted center of Capture A.



4 System Monitoring and Status

The `status.py` module implements the `StatusDevice` class, which performs low-overhead hardware monitoring by reading directly from the Linux virtual filesystems (`/proc` and `/sys`). It captures:

- CPU usage per core from `/proc/stat`.
- RAM and Swap utilization from `/proc/meminfo`.
- SoC temperature from `thermal_zone0`.
- Network latency via ICMP pings.

5 System Initialization

The `init_sys.py` script provisions the environment, creates directories (`Logs`, `Queue`), and resets shared memory.